
1. What is Linear Regression? How does it differ from Logistic Regression?

Linear Regression predicts continuous values by fitting a straight line between input features and output. Logistic Regression is used for classification and predicts probabilities using a sigmoid function. Linear outputs real numbers, while logistic outputs values between 0 and 1. Hence, their use cases differ (regression vs classification).

2. Equation of multiple linear regression (n features)

$$y = \beta_0 + \beta_1x_1 + \beta_2x_2 + \dots + \beta_nx_n + \epsilon$$

This equation represents the relationship between one dependent variable and multiple independent variables. Each coefficient shows the impact of its feature. ϵ is the error term.

3. Cost function in Linear Regression

The cost function is Mean Squared Error (MSE), which measures the average squared difference between actual and predicted values. We minimize it to improve model accuracy. Lower cost means predictions are closer to actual values. It ensures optimal model parameters.

4. Gradient Descent

Gradient Descent is an optimization algorithm used to minimize the cost function. It updates weights iteratively in the direction of the negative gradient. This gradually reduces error. It helps find optimal weights efficiently.

5. R² Score

R² measures how well the model explains variance in the data. It ranges from 0 to 1. A higher value means better fit. Ideal value is close to 1.

6. MAE vs MSE vs RMSE

MAE is the average absolute error and is robust to outliers. MSE squares errors, so large errors are penalized more. RMSE is the square root of MSE and is in original units. RMSE is easier to interpret.

7. Feature Scaling Importance

Feature scaling ensures all features are on the same scale. It helps Gradient Descent converge faster. Without scaling, features with large values dominate. It improves model performance.

8. Multicollinearity

Multicollinearity occurs when independent variables are highly correlated. It makes coefficients unstable and hard to interpret. Small data changes can cause large coefficient changes. It reduces model reliability.

9. Detect & Handle Outliers

Outliers can be detected using box plots or Z-score/IQR methods. They distort model performance. We can remove or cap them. Alternatively, use robust models.

10. Simple vs Multiple Linear Regression

Simple Linear Regression uses one independent variable. Multiple uses more than one feature. Multiple captures more complex relationships. It generally improves accuracy.

11. Positive vs Negative Coefficient

A positive coefficient means the target increases with the feature. A negative coefficient means the target decreases. It shows direction of relationship. Magnitude shows strength.

12. Train-Test Split

Data is split into training and testing sets. Training data builds the model. Testing data evaluates performance. Using same data causes overfitting.

13. Overfitting

Overfitting occurs when the model learns noise instead of patterns. It performs well on training but poorly on test data. It can be detected by large train-test error gap. Regularization helps reduce it.

14. Ridge Regression (L2)

Ridge adds a penalty term proportional to squared coefficients. It reduces model complexity. It prevents overfitting. Unlike linear regression, it shrinks coefficients but doesn't make them zero.

15. Lasso Regression (L1)

Lasso adds penalty using absolute values of coefficients. It can shrink some coefficients to zero. This performs feature selection. It's preferred when we want simpler models.

16. Normal Equation

Normal Equation directly computes optimal weights using matrix operations. It does not require iteration. It is faster for small datasets. Gradient Descent is better for large datasets.

17. Homoscedasticity

It means constant variance of residuals across all predictions. It is an assumption of linear regression. If violated, model predictions become unreliable. It affects accuracy.

18. Residual Plots

Residual plots show errors vs predicted values. Random distribution indicates a good model. Patterns indicate issues like non-linearity. Helps validate assumptions.

19. Bias-Variance Tradeoff

Bias is error due to simplicity; variance is error due to complexity. High bias underfits, high variance overfits. The goal is to balance both. This improves generalization.

20. Cross-Validation (k-fold)

Cross-validation splits data into multiple folds. Each fold is used for testing once. It gives more reliable performance estimates. Better than single split as it uses all data.

21. Preprocessing USA Housing Dataset

Handled missing values and checked data types. Scaled numerical features. Removed irrelevant columns. Checked correlations.

22. Handling 'Address' Column

Address is categorical and not useful directly. It was dropped or encoded. Usually dropped due to high cardinality. It doesn't add predictive value.

23. Intercept β_0

Intercept is the predicted value when all features are zero. It represents baseline prediction. It shifts the regression line. Important for model equation.

24. Highest Correlation Feature

Correlation matrix or heatmap is used. The feature with highest correlation with price is identified. It has strongest influence. Usually income-related features.

25. P-value

P-value measures statistical significance of a feature. Low p-value (<0.05) means feature is important. High p-value means weak contribution. Helps in feature selection.

26. Improve Model (High RMSE)

Use feature engineering or add more data. Apply regularization (Ridge/Lasso). Try non-linear models. Remove outliers.

27. Polynomial Regression

Polynomial regression fits non-linear relationships using polynomial terms. It extends linear regression. Useful when data is curved. Avoids underfitting.

28. Correlation vs Causation

Correlation shows relationship, not cause-effect. Two variables may be related but not causal. Important to avoid wrong conclusions. Domain knowledge is needed.

29. Deploy Model as API

Save model using pickle. Use Flask/FastAPI to create API. Accept input and return predictions. Host on cloud.

30. Ethical Concerns

Models may reinforce bias in pricing. Data privacy is important. Incorrect predictions can impact decisions. Transparency is required.

Here are **concise 3-4 sentence answers** for your CNN/MNIST viva:

1. CNN vs Dense Network

A Convolutional Neural Network (CNN) is designed for image data and uses convolution layers to automatically extract spatial features. Dense networks (fully connected) treat all inputs equally and ignore spatial structure. CNNs are more efficient for images due to parameter sharing. Dense networks are better for tabular data.

2. Convolution Operation (3×3 filter)

Convolution slides a filter over the image and computes dot products to extract features. A 3×3 filter detects local patterns like edges, textures, or corners. It focuses on small regions of the image. Different filters learn different features.

3. Feature Map

A feature map is the output of applying a filter to an input. It highlights detected features. Conv2D(32) produces 32 feature maps. Each corresponds to one filter.

4. ReLU after Convolution

ReLU introduces non-linearity into the model. It replaces negative values with zero. This helps the network learn complex patterns. It also reduces vanishing gradient issues.

5. MaxPooling

MaxPooling reduces spatial dimensions by taking the maximum value in a region. A 2×2 MaxPool halves height and width. It reduces computation and overfitting. It keeps important features.

6. Softmax (MNIST)

Softmax converts outputs into probabilities across 10 classes. It ensures all outputs sum to 1. Used for multiclass classification.

$$\sigma(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

7. Loss Function (Multiclass)

Categorical Cross-Entropy is used. It measures difference between true and predicted probabilities. Lower loss means better prediction.

$$L = - \sum_{i=1}^K y_i \log(\hat{y}_i)$$

8. Dropout (0.5)

Dropout randomly disables neurons during training. With 0.5, half of 256 neurons (~128) are turned off per iteration. It prevents overfitting. It forces the network to learn robust features.

9. Batch Normalisation

BatchNorm normalizes layer inputs to improve stability. It speeds up training. It is placed after convolution and before/after activation. It reduces internal covariate shift.

10. Reshaping MNIST

Images are reshaped to (28,28,1) to include channel dimension. CNN expects 3D input (height, width, channels). MNIST is grayscale, so channel = 1. This ensures compatibility with Conv layers.

11. One-Hot Encoding

It converts label '7' into a vector like [0,0,0,0,0,0,0,1,0,0]. Only the correct class index is 1. Others are 0. Used for multiclass classification.

12. Adam Optimiser

Adam combines momentum and adaptive learning rates. It adjusts learning rate for each parameter. Faster convergence than SGD. Works well in practice.

13. EarlyStopping

EarlyStopping stops training when validation performance stops improving. Patience=5 means wait 5 epochs before stopping. Prevents overfitting. Saves best model.

14. Confusion Matrix

It shows actual vs predicted classes. Rows = actual, columns = predicted. Diagonal values are correct predictions. Off-diagonal shows misclassifications.

15. Precision, Recall, F1

Precision = correct positives / predicted positives. Recall = correct positives / actual positives. F1 is harmonic mean of precision and recall. Useful for imbalance.

16. Increasing Filters

More filters capture more features. It improves representation power. But increases computation. May cause overfitting if too many.

17. Padding ('same' vs 'valid')

'Same' keeps output size equal to input. 'Valid' reduces size (no padding). Same preserves spatial dimensions. Valid is computationally cheaper.

18. Trainable Parameters (Conv2D)

Parameters = (filter_height × filter_width × input_channels + bias) × number_of_filters. For Conv2D(32,3×3) with 1 channel: $(3 \times 3 \times 1 + 1) \times 32 = 320$. Bias adds 1 per filter.

19. Flatten Layer

Flatten converts 2D feature maps into 1D vector. Required before Dense layers. Dense layers need 1D input. It connects CNN to fully connected layers.

20. Training vs Validation Accuracy

Training accuracy is on training data. Validation accuracy is on unseen data. Large gap indicates overfitting. Both should be high and close.

21. Confused MNIST Digits

Digits like 3 & 5 or 4 & 9 are often confused. They have similar shapes. Poor handwriting increases confusion. Model struggles with subtle differences.

22. Data Augmentation

It increases dataset diversity artificially. Techniques: rotation, shifting, zooming. Helps generalization. Reduces overfitting.

23. Receptive Field

It is the region of input image affecting a neuron. Deeper layers have larger receptive fields. Helps capture global patterns. Important for context understanding.

24. Transfer Learning

Using pre-trained models for new tasks. Not very useful for MNIST (simple dataset). Useful for complex datasets. MNIST can be trained from scratch.

25. ReduceLROnPlateau

Reduces learning rate when validation loss stops improving. Helps fine-tune training. Prevents overshooting minima. Improves convergence.

26. Vanishing Gradient

Gradients become very small in deep networks. Slows learning. ReLU avoids saturation. BatchNorm stabilizes gradients.

27. Shallow vs Deep CNN

Shallow CNN has few layers; faster but less powerful. Deep CNN captures complex patterns. But needs more data and computation. Risk of overfitting.

28. Class Imbalance Handling

Use class weights or oversampling. Data augmentation for minority class. Evaluate using F1-score. Avoid bias toward majority class.

29. `model.evaluate()` vs `predict()`

`evaluate()` returns loss and metrics. `predict()` returns output predictions. Evaluate is for performance. Predict is for inference.

30. Real-Time App

Train and save model. Use Flask/Streamlit for backend. Capture input via camera or drawing pad. Predict and display result in real-time.

Here are **crisp 3-4 sentence viva answers** for RNN/LSTM & sentiment analysis:

1. RNN & sequential data

A Recurrent Neural Network (RNN) processes data sequentially by maintaining a hidden state. It uses previous outputs as inputs for the next step. This makes it suitable for time-series or text. It captures temporal dependencies.

2. Vanishing Gradient Problem

Gradients become very small during backpropagation through time. This makes weight updates negligible. It occurs due to repeated multiplication of small values. Hence, RNN fails to learn long-term dependencies.

3. LSTM Architecture (Gates)

LSTM is a special RNN that handles long-term dependencies. It has three gates: forget, input, and output gate. These control what information to keep, update, or discard. It maintains a cell state.

4. Forget Gate & Cell State Equations

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Forget gate decides what to discard. Cell state updates combine old and new information. This helps retain long-term memory.

5. LSTM vs GRU

LSTM has 3 gates; GRU has 2 (update and reset). GRU is simpler and faster. LSTM handles complex dependencies better. GRU needs fewer parameters.

6. Embedding Layer vs One-Hot

Embedding maps words to dense vectors. It captures semantic relationships. One-hot encoding is sparse and high-dimensional. Embeddings are more efficient.

7. Vocabulary Size (10,000)

It defines number of unique words considered. Top 10,000 frequent words are selected. Rare words are ignored. This balances performance and memory.

8. Sequence Padding

Padding ensures all sequences have equal length. Post-padding adds zeros at the end. It preserves initial sequence information. Preferred for LSTM processing.

9. MAX_LEN = 300

Each review is limited to 300 tokens. Longer reviews are truncated. Shorter ones are padded. This standardizes input size.

10. Bidirectional LSTM

It processes sequence in forward and backward directions. Captures past and future context. Improves understanding of sentence meaning. Useful in sentiment analysis.

11. Binary Cross-Entropy

Used for binary classification tasks. Measures difference between actual and predicted probability.

$$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

12. Sigmoid Activation

Sigmoid maps output to range (0,1). It represents probability. Used in binary classification output layer. Easy to interpret.

13. SpatialDropout1D

Drops entire embedding channels instead of individual values. Maintains sequence structure. Regular dropout drops random neurons. SpatialDropout is better for text data.

14. Teacher Forcing

Uses actual output as next input during training. Helps faster convergence. Not used during inference. In inference, model uses its own predictions.

15. AUC-ROC

Measures model's ability to distinguish classes. Range is 0 to 1. AUC = 0.92 means excellent performance. High true positive, low false positive rate.

16. Decision Threshold

Default threshold is 0.5. Lower it to increase recall. Useful when false negatives are costly. Adjust based on use-case.

17. Precision vs Recall (Example)

Precision = correct positive predictions / predicted positives. Recall = correct positives / actual positives. In sentiment, precision means correct positive reviews predicted. Recall means capturing all actual positives.

18. Recurrent Dropout

Applied to recurrent connections in LSTM. Helps prevent overfitting in sequence

memory. Regular dropout affects input/output layers. Recurrent dropout maintains temporal consistency.

19. Attention Mechanism

Attention focuses on important words in a sequence. Assigns weights to tokens. Improves model performance. Helps capture context better.

20. BERT vs LSTM

BERT is transformer-based and uses attention. It captures bidirectional context better. Pretrained on large datasets. Outperforms LSTM in NLP tasks.

21. Text Tokenisation

Converts text into tokens (words or indices). Removes punctuation, lowercases text. Maps words to integers. Prepares data for embedding.

22. Word vs Character Models

Word-level uses words as tokens. Character-level uses individual characters. Word models are faster. Character models handle spelling variations better.

23. Handling Negation

Use n-grams (e.g., "not good"). Capture context with LSTM/BERT. Negation flips sentiment meaning. Important for accuracy.

24. ReduceLROnPlateau

Reduces learning rate when validation loss stops improving. Helps fine-tune training. Avoids overshooting minima. Improves convergence.

25. SMOTE

Synthetic Minority Over-sampling Technique. Creates synthetic samples for minority class. Used in imbalanced datasets. Not needed if IMDB is balanced.

26. 5-Star Rating Extension

Convert binary to multiclass classification. Use Softmax output layer. Change loss to categorical cross-entropy. Predict 5 classes.

27. Doubling LSTM Units

Increases model capacity. Learns more complex patterns. But increases computation and overfitting risk. Needs more data.

28. Training vs Validation Loss Graph

If both decrease → good learning. If training decreases but validation increases → overfitting. Helps diagnose model performance. Used for tuning.

29. Output 0.82 Meaning

Model predicts 82% probability of positive sentiment. Since >0.5 , classified as positive. Higher confidence in prediction. Threshold can be adjusted.

30. Limitations of LSTM (Long Docs)

LSTM struggles with very long sequences. Memory and computation increase. Long-range dependencies still hard. Transformers handle long text better.

Here are **concise 3-4 sentence answers** for your viva:

1. Hyperparameter tuning vs training

Hyperparameter tuning is selecting the best configuration (e.g., learning rate, batch size, filters) before/during training. Training learns model parameters (weights and biases) from data. Hyperparameters are set manually or via search methods. Parameters are learned automatically.

2. Learning Rate effect

Learning rate controls step size during optimization. At $LR = 0.1$, updates may overshoot and cause instability. At 0.0001 , learning is very slow. A balanced LR ensures fast and stable convergence.

3. SGD vs Adam vs RMSprop

SGD is simple but slow. RMSprop adapts learning rate for each parameter. Adam combines momentum and adaptive learning. Adam usually works best due to faster convergence.

4. Filter Size Effect

3×3 captures fine details like edges. 5×5 and 7×7 capture larger patterns. Larger filters increase computation. Smaller filters are preferred in deep CNNs.

5. More Conv Layers

Increasing layers improves feature hierarchy. Early layers detect edges; deeper layers detect complex patterns. But too many layers may cause overfitting. Requires more computation.

6. Transfer Learning

Using pre-trained models for new tasks. Examples: ResNet, VGG16. Used when dataset is small. Saves training time and improves performance.

7. Fine-Tuning

Fine-tuning means unfreezing some pre-trained layers. Allows model to adapt to new data. Applied after initial training. Improves accuracy.

8. Data Augmentation

Techniques: rotation (rotate image), flipping (mirror image), zoom (scale), shifting (translate), brightness (change intensity). Increases data diversity. Reduces overfitting.

9. ImageDataGenerator

It generates augmented image batches during training. Configured with parameters like `rotation_range`, `zoom_range`, `horizontal_flip`. Applied only to training data. Improves generalization.

10. Dropout (0.4)

Dropout randomly disables neurons. With 0.4, 40% of 512 (~205) neurons are dropped. Prevents overfitting. Encourages robust learning.

11. L2 Regularisation

Adds penalty proportional to squared weights. Prevents large weights. Reduces overfitting. Improves generalization.

12. GlobalAveragePooling2D

It averages each feature map into one value. Reduces parameters. Preferred over Flatten for reducing overfitting. Used in modern CNNs.

13. Receptive Field

It is the input region influencing a neuron. It grows with depth. Deeper layers see larger context. Important for capturing global patterns.

14. Grid Search

Tests all combinations of hyperparameters. Exhaustive but expensive. Used for small search spaces. Ensures best combination.

15. Random Search

Randomly samples hyperparameters. More efficient than Grid Search. Covers wider space. Often finds good results faster.

16. Overfitting vs Underfitting

Overfitting: high training accuracy, low validation accuracy. Underfitting: both accuracies low. Training curves help detect this. Gap indicates overfitting.

17. CAM

Class Activation Mapping highlights important regions. Shows which parts influence prediction. Helps interpret CNN. Useful for debugging.

18. Separate Validation Set

Used to tune hyperparameters. Prevents bias in evaluation. Test set must remain unseen. Ensures fair performance estimate.

19. Batch Size

Batch size is number of samples per update. Small batch (32) gives stable generalization. Large batch (256) speeds training but may reduce accuracy. Trade-off exists.

20. Minima & Saddle Points

Global minima is best solution. Local minima is suboptimal solution. Saddle point has zero gradient but not minimum. Optimization algorithms handle these.

21. Weight Initialisation

Initial weights affect training speed and stability. Poor initialization causes vanishing/exploding gradients. Methods like Xavier/He are used. Important for deep networks.

22. Learning Rate Scheduler

Adjusts learning rate during training. ReduceLROnPlateau reduces LR when validation loss stops improving. Helps fine-tune model. Improves convergence.

23. Freezing Base Layers

Pre-trained layers contain useful features. Freezing prevents them from changing. Avoids overfitting early. Later, some layers can be unfrozen.

24. Metrics for Imbalanced Data

Use Precision, Recall, F1-score, AUC-ROC. Accuracy can be misleading. Focus on minority class performance. Important in medical data.

25. Prevent Data Leakage

Split data before augmentation. Apply augmentation only to training set. Do not leak test data info. Ensures fair evaluation.

26. Sigmoid in Binary Classifier

Outputs probability between 0 and 1. Used for binary classification. Easy thresholding. Interpretable output.

27. Grad-CAM vs CAM

Grad-CAM uses gradients for visualization. Works with more architectures. CAM requires specific structure. Grad-CAM is more flexible.

28. Small Dataset (500 images)

Use transfer learning. Apply heavy augmentation. Use cross-validation. Avoid training from scratch.

29. Confusion Matrix (Binary)

TP: correct positive. TN: correct negative. FP: false positive. FN: false negative. Used to compute metrics.

30. Hardware for CNNs

GPUs or TPUs are preferred. They speed up matrix operations. Faster training than CPUs. Essential for deep learning.

Here are **clear 3-4 sentence viva answers** for *Assignment 3: LSTM - Sentiment Analysis (IMDB)*:

1. RNN & sequential data

RNN processes data step-by-step while maintaining a hidden state. It uses past information to influence current predictions. This makes it ideal for sequential data like text. It captures temporal dependencies.

2. Vanishing Gradient Problem

Gradients shrink during backpropagation through many time steps. This leads to very small weight updates. It occurs due to repeated multiplication of values <1 . Hence, long-term dependencies are not learned.

3. LSTM Architecture (Gates)

LSTM improves RNN by adding memory cells. It has three gates: forget, input, and output gate. These control information flow. It preserves long-term dependencies.

4. Forget Gate & Cell State

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Forget gate decides what to remove. Cell state combines old and new information. This maintains memory.

5. LSTM vs GRU

LSTM has more gates and is more powerful. GRU is simpler and faster. LSTM handles complex sequences better. GRU needs fewer parameters.

6. Embedding vs One-Hot

Embedding converts words into dense vectors. It captures semantic meaning. One-hot is sparse and high-dimensional. Embedding is more efficient.

7. Vocabulary Size (10,000)

It defines number of unique words used. Top 10,000 frequent words are selected. Rare words are ignored. This balances accuracy and memory.

8. Sequence Padding

Padding makes all sequences equal length. Post-padding adds zeros at the end. It preserves initial important words. Suitable for LSTM.

9. MAX_LEN = 300

Each review is limited to 300 tokens. Longer reviews are truncated. Short ones are padded. Ensures fixed input size.

10. Bidirectional LSTM

Processes sequence forward and backward. Captures full context. Improves understanding of sentiment. Useful for text tasks.

11. Binary Cross-Entropy

Used for binary classification. Measures difference between true and predicted values.

$$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

12. Sigmoid Activation

Maps output to (0,1). Represents probability. Used in binary output layer. Easy to interpret.

13. SpatialDropout1D

Drops entire feature channels in embeddings. Keeps sequence structure intact. Regular dropout drops individual values. Better for text.

14. Teacher Forcing

Uses actual output as next input during training. Speeds up learning. Not used during inference. Model uses its own predictions then.

15. AUC-ROC

Measures classification performance. $AUC = 0.92$ means excellent separation. High true positives, low false positives. Closer to 1 is better.

16. Decision Threshold

Default is 0.5. Lower it to increase recall. Useful when missing positives is costly. Adjust based on application.

17. Precision vs Recall

Precision: correct positives among predicted positives. Recall: correct positives among actual positives. In sentiment, recall ensures all positive reviews are captured. Precision ensures predictions are correct.

18. Recurrent Dropout

Applied on recurrent connections. Helps prevent overfitting in sequence learning. Regular dropout is on inputs/outputs. Maintains temporal consistency.

19. Attention Mechanism

Focuses on important words in a sentence. Assigns weights to tokens. Improves model performance. Helps capture context better.

20. BERT vs LSTM

BERT uses transformers and attention. Captures bidirectional context better. Pretrained on large data. Performs better than LSTM.

21. Text Tokenisation

Converts text into tokens. Removes punctuation, lowercases text. Maps words to indices. Prepares for embedding.

22. Word vs Character Models

Word-level uses words as units. Character-level uses characters. Word models are faster. Character models handle spelling errors.

23. Handling Negation

Use n-grams like "not good". LSTM captures context. Negation flips sentiment. Important for accuracy.

24. ReduceLROnPlateau

Reduces learning rate when validation loss stops improving. Helps fine-tune training. Prevents overshooting. Improves convergence.

25. SMOTE

Generates synthetic minority samples. Used for imbalanced datasets. Not needed if IMDB is balanced. Helps improve recall.

26. 5-Star Predictor

Convert to multiclass classification. Use Softmax output. Change loss to categorical cross-entropy. Predict 5 classes.

27. Doubling LSTM Units

Increases model capacity. Learns complex patterns. Increases computation and overfitting risk. Needs more data.

28. Training vs Validation Loss

Both decreasing → good model. Validation increasing → overfitting. Helps in tuning. Used to stop training.

29. Output 0.82 Meaning

Model predicts 82% probability of positive sentiment. Since >0.5 , classified as positive. High confidence. Threshold adjustable.

30. LSTM Limitations

Struggles with very long sequences. High computation cost. Memory limitations. Transformers perform better.

Here are **clear 3–4 sentence viva answers** for *Assignment 5: Data Classification Algorithms*:

1. Classification vs Regression

Classification predicts discrete class labels (e.g., spam/not spam). Regression predicts continuous values (e.g., price). Classification outputs categories, while regression outputs numbers. Both are supervised learning tasks.

2. Decision Tree

A Decision Tree splits data based on feature values. At each node, it selects the best feature using a criterion like Information Gain. The goal is to create pure subsets. Leaves represent final class predictions.

3. Information Gain

Information Gain measures reduction in entropy after a split. It helps choose the best feature. Higher gain means better split. Used in tree construction.

4. Gini Impurity

$$Gini = 1 - \sum p_i^2$$

Gini measures how often a randomly chosen element would be misclassified. Lower Gini means better purity. Entropy uses log, Gini is faster. Both are used for splitting.

5. Naive Bayes

Naive Bayes is a probabilistic classifier based on Bayes' theorem. It assumes features are independent. This assumption makes it "naive." Works well for text classification.

6. Bayes' Theorem

$$P(C|X) = \frac{P(X|C)P(C)}{P(X)}$$

$P(C|X)$ is posterior probability. $P(X|C)$ is likelihood. $P(C)$ is prior. Used to classify data based on probability.

7. KNN Classifier

KNN stores training data. It predicts based on majority class of K nearest points. Distance metrics like Euclidean are used. It is a lazy learner.

8. Choosing K in KNN

Small K $\rightarrow$ sensitive to noise (overfitting). Large K $\rightarrow$ smoother but may underfit. Optimal K is chosen via cross-validation. Usually odd number to avoid ties.

9. SVM & Hyperplane

SVM finds the best boundary (hyperplane) separating classes. It maximizes margin between classes. Hyperplane is decision boundary. Works well for high-dimensional data.

10. Kernel Trick

Transforms data into higher dimension. Allows non-linear separation. Common kernels: linear, polynomial, RBF. Avoids explicit transformation.

11. Random Forest

Ensemble of multiple decision trees. Each tree is trained on random data subset. Final prediction is majority vote. Reduces overfitting.

12. Bagging

Bootstrap Aggregating uses random sampling with replacement. Each model trains on different subset. Reduces variance. Used in Random Forest.

13. Confusion Matrix

TP: correct positive, TN: correct negative. FP: predicted positive but wrong. FN: missed positive. Used to evaluate classifier.

14. Accuracy

Accuracy = correct predictions / total predictions. Misleading in imbalanced data. High accuracy may hide poor minority class performance. Use other metrics.

15. Precision vs Recall

Precision: correctness of positive predictions. Recall: ability to find all positives.

Use precision when false positives are costly. Use recall when missing positives is critical.

16. F1-Score

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

F1 balances precision and recall. Useful in imbalanced datasets. Better than accuracy when classes are uneven.

17. ROC Curve & AUC

ROC plots TPR vs FPR. AUC measures overall performance. Value closer to 1 is better. Shows classification ability.

18. Cross-Validation (10-fold)

Data split into 10 parts. Each part used as test once. Remaining used for training. Results averaged. Gives reliable estimate.

19. Overfitting in Decision Tree

Tree becomes too complex. Memorizes training data. Pruning removes unnecessary branches. `max_depth` limits tree size.

20. Feature Importance (Random Forest)

Measures contribution of each feature. Based on impurity reduction. Higher importance means more influence. Helps feature selection.

21. Hard vs Soft Margin SVM

Hard margin: no misclassification allowed. Soft margin: allows some errors. Soft margin is more practical. Handles noise.

22. Stratified Sampling

Maintains class proportions in splits. Ensures balanced training/testing. Important for imbalanced data. Improves evaluation.

23. Label vs One-Hot Encoding

Label encoding assigns numbers. One-hot creates binary vectors. Use label for ordinal data. Use one-hot for nominal data.

24. Gradient Boosting

Builds models sequentially. Each corrects previous errors. Focuses on hard samples. Different from Random Forest (parallel).

25. XGBoost

Optimized gradient boosting algorithm. Faster and more accurate. Handles missing values. Popular in BI tasks.

26. Class Imbalance

Model becomes biased toward majority class. Handle using resampling or class weights. Use proper metrics like F1. Improves fairness.

27. Learning Curve

Shows training vs validation performance. Helps detect overfitting/underfitting. Guides model improvement. Useful for tuning.

28. Hyperparameter Tuning (Decision Tree)

Tune parameters like max_depth, min_samples_split, criterion. Improves performance. Done using Grid/Random search. Avoids overfitting.

29. WEKA

WEKA is a data mining tool. Provides *GUI* for *ML* algorithms. Used to apply classification easily. No coding required.

30. Explaining Model to Manager

Use simple language and visuals. Explain key factors influencing decisions. Use examples instead of equations. Focus on business impact.
